

# WC2026 Dashboard — Architecture & Documentation

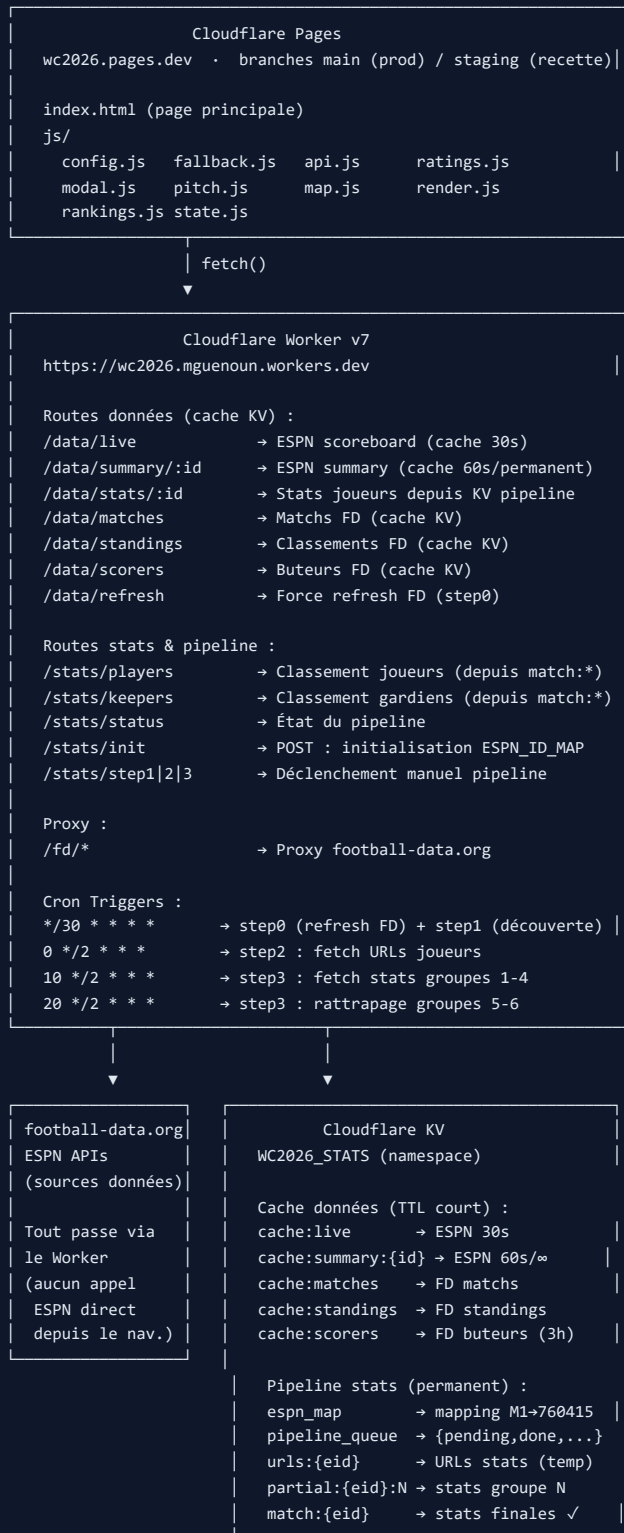
---

## Vue d'ensemble

Dashboard de suivi de la Coupe du Monde 2026 (USA/Canada/Mexique), déployé en tant que site statique sur **Cloudflare Pages** (branches `main` → prod, `staging` → recette) avec un backend serverless sur **Cloudflare Workers** (v7) et un cache persistant **Cloudflare KV**.

---

## Architecture globale



# Composants GitHub Pages

## Structure des fichiers

```
wc2026/
├─ index.html          # Page principale (Cloudflare Pages)
├─ js/
│   ├── config.js      v2 # Constantes, maps, utilitaires
│   ├── fallback.js    v3 # Données statiques matchs (63 matchs + KO)
│   ├── api.js         v10 # Fetch Worker, traitement scores live
│   ├── predictions.js v5 # Modèle Poisson v4 (prédictions matchs à venir)
│   ├── ratings.js     v4 # Algo rating joueurs v8 (sans xG/xA)
│   ├── modal.js       v7 # Modales stats match + lien YouTube par but
│   ├── pitch.js       v4 # Terrain SVG, compositions, openLineupESPN
│   ├── map.js         v6 # Vue liste matchs + prédictions score + ► YouTube
│   ├── render.js      v10 # Timelines, bracket KO, Meilleurs 3èmes
│   ├── rankings.js    v3 # Gardiens, buteurs, classement joueurs (avec Min)
│   └─ state.js        v6 # renderAll, switchView (7 vues), scheduleRefresh
```

## Ordre de chargement des scripts (critique)

```
<script src="js/config.js?v=2"></script>      <!-- 1. Constantes globales -->
<script src="js/fallback.js?v=3"></script>     <!-- 2. Données statiques -->
<script src="js/api.js?v=10"></script>         <!-- 3. Fetch Worker/FD -->
<script src="js/predictions.js?v=5"></script>   <!-- 4. Modèle Poisson v4 -->
<script src="js/ratings.js?v=4"></script>       <!-- 5. Algo rating v8 -->
<script src="js/modal.js?v=7"></script>         <!-- 6. Modales + YouTube par but -->
<script src="js/pitch.js?v=4"></script>         <!-- 7. Terrain SVG -->
<script src="js/map.js?v=6"></script>          <!-- 8. Vue liste + prédictions -->
<script src="js/render.js?v=10"></script>       <!-- 9. Rendu groupes + KO + Meilleurs 3èmes -->
<script src="js/rankings.js?v=3"></script>      <!-- 10. Classements joueurs -->
<script src="js/state.js?v=6"></script>        <!-- 11. Init + orchestration (7 vues) -->
```

## Variables globales clés (config.js)

| Variable      | Description                                                          |
|---------------|----------------------------------------------------------------------|
| PROXY_BASE    | URL du Worker ( <code>https://wc2026.mguenoun.workers.dev</code> )   |
| ESPN_ID_MAP   | Mapping matchKey → ESPN event ID (ex: <code>'M1': '760415'</code> )  |
| TEAM_MAP      | Noms d'équipes ESPN → noms français                                  |
| GC            | Couleurs par groupe ( <code>{A: '#color', B: '#color', ...}</code> ) |
| allMatches    | Array des matchs (peuplé par <code>loadFallback()</code> )           |
| standings     | Classements par groupe (peuplé par <code>fetchAll()</code> )         |
| scorers       | Liste des buteurs (peuplé par <code>fetchAll()</code> )              |
| activeFilter  | Filtre groupe actif ('all' ou lettre)                                |
| _currentMatch | Match actuellement ouvert en modale                                  |
| DISPLAY_TZ    | Timezone d'affichage ( <code>Africa/Casablanca</code> )              |

# Cloudflare Worker v7 (worker.js)

## Step 0 — Refresh données FD (toutes les 30min avec step1)

- Fetch matchs FD → écrit `cache:matches` (différentiel, seulement si changement)
- Si changement détecté → fetch standings FD → écrit `cache:standings`
- Fetch buteurs FD toutes les 3h → écrit `cache:scorers`

Routes cache KV (front → Worker → KV)

| Route             | Source                   | TTL                               |
|-------------------|--------------------------|-----------------------------------|
| /data/live        | ESPN scoreboard J-1 + J  | 30s                               |
| /data/summary/:id | ESPN summary             | 60s si live, permanent si terminé |
| /data/stats/:id   | KV match:{id} (pipeline) | Permanent                         |
| /data/matches     | KV cache:matches         | ~30min (step0)                    |
| /data/standings   | KV cache:standings       | ~30min si changement (step0)      |
| /data/scorers     | KV cache:scorers         | 3h (step0)                        |

Pipeline de calcul des stats joueurs

Le plan gratuit Cloudflare limite chaque invocation à 50 sous-requêtes HTTP. Le pipeline découpe le traitement en 3 étapes :

Step 1 — Découverte (~8 sous-requêtes)

- Scan ESPN scoreboards sur les 7 derniers jours
- Détecte les nouveaux matchs terminés → alimente espn\_map
- Compare avec pipeline\_queue → ajoute les ESPN IDs non traités dans queue.pending

Step 2 — Fetch URLs (~7 sous-requêtes)

- Prend le premier match pending
- Fetch ESPN summary (1) + competitors (1) + 2 rosters (2×2)
- Extrait les URLs de stats individuelles (22-52 joueurs)
- Stocke dans KV["urls:{eid}"]
- Déplace en queue.processing

Step 3 — Fetch stats par groupes de 10 (~12-42 sous-requêtes)

- Lit KV["urls:{eid}"]
- Fetch les stats de 10 joueurs en SÉQUENCE (pas Promise.all)
- Maximum 4 groupes par invocation (40 appels max, sous la limite de 50)
- Stocke dans KV["partial:{eid}:{N}"]
- Quand tous les groupes sont prêts → assemble dans KV["match:{eid}"]

KV Keys

| Clé                 | Contenu                                | Durée de vie                           |
|---------------------|----------------------------------------|----------------------------------------|
| cache:live          | Events ESPN scoreboard J-1+J           | 30s                                    |
| cache:summary:{eid} | ESPN summary complet                   | 60s (live) / permanent (FT)            |
| cache:matches       | Matches FD (différentiel)              | Mis à jour par step0 (~30min)          |
| cache:standings     | Classements FD                         | Mis à jour par step0 si changement     |
| cache:scorers       | Buteurs FD top 20                      | Mis à jour toutes les 3h               |
| espn_map            | Mapping matchKey → ESPN ID + infos     | Permanent                              |
| pipeline_queue      | { pending:[], processing:[], done:[] } | Permanent                              |
| urls:{eid}          | URLs stats joueurs + infos match       | Temporaire (supprimé après assemblage) |
| partial:{eid}:{N}   | Stats du groupe N de joueurs           | Temporaire (supprimé après assemblage) |
| match:{eid}         | Stats finales de tous les joueurs      | Permanent                              |

| Nom          | Description                              |
|--------------|------------------------------------------|
| API_TOKEN_FD | Clé API football-data.org                |
| API_TOKEN    | Clé API football-data.org (alias legacy) |

KV Namespace Binding

| Variable name | KV Namespace | Namespace ID                     |
|---------------|--------------|----------------------------------|
| STATS_KV      | WC2026_STATS | 749700d94d114156a758b14e2e9b6587 |

Cron Triggers

| Expression                    | Fréquence             | Rôle                                           |
|-------------------------------|-----------------------|------------------------------------------------|
| <code>* / 30 * * * *</code>   | Toutes les 30min      | step0 (refresh FD) + step1 (détection matches) |
| <code>0 * / 2 * * * *</code>  | Toutes les 2h         | step2 : fetch URLs du prochain match pending   |
| <code>10 * / 2 * * * *</code> | Toutes les 2h + 10min | step3 : fetch stats groupes 1-4                |
| <code>20 * / 2 * * * *</code> | Toutes les 2h + 20min | step3 : rattrapage groupes 5-6                 |

Sources de données

| Source                                                 | Usage                                  | Accès                                                               |
|--------------------------------------------------------|----------------------------------------|---------------------------------------------------------------------|
| <b>ESPN</b> <code>site.api.espn.com</code>             | Scores live, compositions, stats match | Via Worker <code>/data/live</code> , <code>/data/summary/:id</code> |
| <b>ESPN Core</b> <code>sports.core.api.espn.com</code> | Stats détaillées joueurs (pipeline)    | Direct depuis Worker (step2/step3)                                  |
| <b>football-data.org</b>                               | Matches, standings, buteurs            | Via Worker <code>/fd/*</code> + cache KV                            |

Aucun appel ESPN n'est fait directement depuis le navigateur. Tout passe par le Worker, qui maintient un cache KV pour limiter les requêtes upstream.

Algo Rating Joueurs (v8)

Formule par poste avec architecture `base + volScore + offScore` :

- Base** : 6.3 pour tous les postes
- Per90 atténué** : `1 + (90/min - 1) × 0.2` (évite la distorsion sur peu de minutes)
- progCarries** plafonné à 6 avant normalisation
- volScore** (métriques de volume/défense) : plafonné selon poste (1.2 CM → 1.8 DEF/FB/AM)
- offScore** (buts, assists, tirs cadrés) : plafonné à 2.5 (2.8 pour FW)
- Carton rouge** : -1.0 / **Carton jaune** : -0.3
- Plage** : [4.0 — 9.5]
- Sans xG/xA** : supprimés en v8 (causaient une inflation de +0.1 à +0.5 pts sur AM/FW vs SofaScore/FotMob)

Poids par rôle :

| Rôle | Vol principal                     | Off principal                       | Pénalité           |
|------|-----------------------------------|-------------------------------------|--------------------|
| GK   | saves×0.35, cs×0.30               | goals×2.0                           | —                  |
| DEF  | tackles×0.15, duels×0.80, cs×0.35 | goals×1.8                           | —                  |
| FB   | crosses×0.15, duels×0.65, cs×0.25 | goals×1.6, assists×1.0              | —                  |
| DM   | tackles×0.18, passes×0.60         | goals×1.5                           | —                  |
| CM   | passes×0.45, duels×0.45           | goals×1.4, shots×0.15               | —                  |
| AM   | shots×0.20, progCarries×0.08      | goals×1.4, assists×0.90             | -0.15 si shots<0.5 |
| FW   | —                                 | goals×1.5, shots×0.22, assists×0.80 | -0.20 si shots<0.5 |

## Prédictions de Score — Modèle Poisson v4 (predictions.js)

Les prédictions sont calculées côté navigateur pour chaque match à **venir** (non démarré, non terminé, sans placeholder KO).

### Prior FIFA

Chaque équipe dispose d'un **rating FIFA officiel** (juin 2026, 48 équipes). En l'absence de stats de tournoi, le modèle revient à ce prior :

```
priorGF_A = avgGF × (fifaA / avgFifa)^1.5
priorGA_A = avgGA × (avgFifa / fifaA)^1.5
```

- `avgGF` / `avgGA` : buts par équipe par match calculés depuis les classements en cours
- `avgFifa` ≈ 1580 (moyenne des 48 équipes)
- Exposant 1.5 : différencie suffisamment sans exagérer (Maroc vs Écosse → ~58% favori)

### Régression bayésienne K=6

Après chaque match joué, les stats réelles ne pèsent que `w = played / (played + K)` :

```
w = played / (played + 6)
gf_A = w × rawGF_A + (1-w) × priorGF_A
ga_A = w × rawGA_A + (1-w) × priorGA_A
```

Avec K=6, après 1 match joué le poids sur les stats réelles est seulement 14%. Cela évite qu'une défaite difficile (ex: 0-2 vs Brésil J1) écrase le prior d'une équipe de niveau mondial.

### Paramètres Poisson

```
lambdaA = clip(gf_A × ga_B / avgGF, 0.2, 4.0)
lambdaB = clip(gf_B × ga_A / avgGF, 0.2, 4.0)
```

La distribution de Poisson bivarié  $P(i \text{ buts A}, j \text{ buts B}) = \text{PMF}(\lambda_A, i) \times \text{PMF}(\lambda_B, j)$  est calculée sur une grille 8×8 (0 à 7 buts chaque côté).

### Argmax par catégorie

Les probabilités sont séparées en trois catégories :

- **V (Victoire A)** : somme des  $P(i,j)$  avec  $i > j$
- **N (Nul)** : somme des  $P(i,j)$  avec  $i = j$
- **D (Défaite A)** : somme des  $P(i,j)$  avec  $i < j$

Le **score affiché** est le score  $(i,j)$  maximisant  $P$  dans la catégorie la plus probable (pas le score global le plus probable, ce qui permettrait d'afficher un nul 0-0 même quand V=55%).

### Affichage (map.js v6)

Pour chaque match à venir, la ligne affiche :

-  X-Y en orange (stats disponibles) ou gris (prior FIFA uniquement)
- Vxx% · Nxx% · Dxx% colorisés : vert si favori, rouge si perdant, gris si nul prédit
- Tooltip : Poisson  $\lambda=X.X / Y.Y$

`buildPredictions()` est appelé depuis `api.js` après chaque `fetchAll()` et après chaque mise à jour de standings en live.

## Classement Joueurs (rankings.js v3 + Worker /stats/players)

### Pipeline de calcul (Worker)

`handlePlayers()` agrège toutes les clés `match:{eid}` du KV pour construire le classement :

```
Pour chaque joueur sur tous les matchs :
totalMinutes += minutes_jouées_ce_match
totalRating  += rating_ce_match x minutes_jouées_ce_match
goals        += buts
assists      += passes décisives
matches      += 1

rating_final = totalRating / totalMinutes  (moyenne pondérée par minutes)
```

Le classement est trié par `rating_final` décroissant.

### Algo Rating v8 (ratings.js)

Formule par poste : `rating = 6.3 + volScore + offScore`

| Métrique      | Description                                                                                    |
|---------------|------------------------------------------------------------------------------------------------|
| Per90 atténué | <code>1 + (90/min - 1) x 0.2</code> — atténue la distorsion sur peu de minutes                 |
| volScore      | Métriques de volume/défense (tackles, passes, duels, saves, CS) — plafonné 1.2→1.8 selon poste |
| offScore      | Buts, assists, tirs cadrés — plafonné à 2.5 (2.8 FW)                                           |
| Carton rouge  | -1.0                                                                                           |
| Carton jaune  | -0.3                                                                                           |
| Plage finale  | [4.0 — 9.5]                                                                                    |

Sans xG/xA (supprimés en v8 car causaient une inflation de +0.1 à +0.5 sur AM/FW vs SofaScore).

### Colonnes affichées

| Colonne                                                                            | Description                                               |
|------------------------------------------------------------------------------------|-----------------------------------------------------------|
| MJ                                                                                 | Matches joués (cumul)                                     |
| Min                                                                                | Minutes cumulées sur tous les matchs                      |
|  | Buts totaux (cumul)                                       |
| →                                                                                  | Passes décisives totales (cumul)                          |
| Moy.                                                                               | Note moyenne pondérée par minutes — critère de classement |

## Refresh des scores live

`state.js` — `scheduleRefresh()` toutes les **30s** (setTimeout récursif) :

```

Tick N :
  hasLive = allMatches.some(m => m.isLive) ← évalué au moment du tick
  Si hasLive → fetchESPNLiveScores() [Worker /data/live, cache 30s]
  Si tick%5 == 0 → fetchAll() [refresh complet toutes les 2.5min]
  Sinon tick%10==0 → fetchAll() [refresh complet toutes les 5min]

```

`processMatches()` — gestion décalage UTC/heure locale :

- Matching ESPN events sur `dayKey` du match (heure locale Paris)
- Fallback J-1 si le match était après minuit UTC la veille (ex: match 21h Paris = J-1 UTC)

## Bracket KO et Meilleurs 3èmes (render.js / fallback.js)

- `fallback.js` : données statiques des 63 matchs (GS + KO), placeholders `1er Gr.X`, `2e Gr.X`, `3e A/B/C/D`, `V Mxx`, `Vainq./Perdant`
- `resolveKOTeam(placeholder)` : résout les placeholders depuis `allMatches` et `standings`
- Noms résolus affichés entre parenthèses dans la vue liste (`map.js`) et dans le bracket (`render.js`)
- `getAll3rd()` : collecte le 3e de chaque groupe ayant joué  $\geq 1$  match, trié par pts/DB/BP. Détecte les cas ex æquo (2e et 3e avec stats identiques) et renseigne un champ `coTeam`.
- `buildThirdAssign()` : matching bipartite (chemin augmentant) pour assigner chaque 3e qualifié à un slot KO. Chaque slot liste les groupes éligibles selon la règle FIFA ; l'algo garantit un matching maximal même quand le greedy simple échouerait.
- `renderThirds()` : onglet dédié affichant les 12 troisièmes classés, top 8 qualifiés mis en évidence (✓), badge `prov.` si le groupe n'a pas fini, badge `ex æq.` si deux équipes sont parfaitement à égalité.
- En cas d'ex æquo, la valeur dans la Map `_thirdAssign` contient les deux noms (ex: "Maroc / Brésil") → affiché tel quel dans la parenthèse du bracket/liste KO.

## Initialisation (one-shot depuis la console navigateur)

À lancer une seule fois après déploiement ou `forceReset` :

```

(async function() {
  var mapping = {};
  allMatches.filter(m => m.isFT && ESPN_ID_MAP[m.id]).forEach(function(m) {
    mapping[ESPN_ID_MAP[m.id]] = {
      espnId: ESPN_ID_MAP[m.id], matchKey: m.id,
      t1: m.t1, t2: m.t2, score: m.score, dayKey: m.dayKey
    };
  });
  var r = await fetch('https://wc2026.mguenoun.workers.dev/stats/init', {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ mapping, forceReset: true })
  });
  console.log('Init:', await r.json());
})();

```

```

// Pipeline complet (step2+step3 en boucle)
(async function runPipeline() {
  var base = 'https://wc2026.mguenoun.workers.dev';
  var h = { headers: { Origin: location.origin } };
  for (var i = 0; i < 200; i++) {
    var s = await fetch(base + '/stats/status', h).then(r => r.json());
    if (s.queue.pending.length === 0 && s.queue.processing.length === 0) {
      console.log('Termine', s.kv.matches, 'matches'); break;
    }
    if (s.queue.pending.length > 0) await fetch(base + '/stats/step2', h);
    await fetch(base + '/stats/step3', h);
    await new Promise(res => setTimeout(res, 500));
  }
})();

```



## Reset KV ratings (si recalibration formule)

```
# Dans worker/ (Git Bash) – pipeline_queue via fichier (wrangler strip les guillemets en CLI)
echo '{"pending":[],"processing":[],"done":[]}' > /tmp/pq.json
npx wrangler kv key put --remote --binding=STATS_KV "pipeline_queue" --path /tmp/pq.json

# Supprimer les clés match:* une par une
npx wrangler kv key list --remote --binding=STATS_KV --prefix="match:"
npx wrangler kv key delete --remote --binding=STATS_KV "match:<id>"

# Relancer step1 puis pipeline
curl -H "Origin: http://localhost" https://wc2026.mguenoun.workers.dev/stats/step1
```

## Monitoring

```
fetch('https://wc2026.mguenoun.workers.dev/stats/status')
  .then(r => r.json()).then(d => console.log(JSON.stringify(d, null, 2)));
```

Cloudflare Dashboard → worker `wc2026` → **Observability** → **Logs** : filtrer sur `scheduled` .

## Structure du repo GitHub

```
mguenoun/wc2026 (repo)
├─ index.html          # Cloudflare Pages (main → prod, staging → recette)
├─ js/
│  └─ config.js        v2
│  └─ fallback.js      v3
│  └─ api.js           v10
│  └─ predictions.js   v5 # Modèle Poisson v4 + prior FIFA
│  └─ ratings.js       v4
│  └─ modal.js         v7 # Lien YouTube par but
│  └─ pitch.js         v4
│  └─ map.js           v6 # Prédications score + ► YouTube résumé
│  └─ render.js        v10 # Meilleurs 3èmes + matching bipartite + ex æquo
│  └─ rankings.js      v3 # Min cumulées + Moy. pondérée
│  └─ state.js         v6 # 7 vues : groups/standings/thirds/knockout/scorers/keepers/players
├─ worker/
│  └─ worker.js        v7 # Deploye via wrangler deploy
├─ tests/
│  └─ browser/
│     └─ processMatches.test.js # normTeam, processMatches, computeStandings + pos
│     └─ render.test.js        # getAll3rd (ex æquo), resolveKOTeam, buildThirdAssign
│  └─ worker/
│     └─ calcRating.test.js
├─ docs/
│  └─ diag1_architecture.{svg,png}
│  └─ diag2_sequence.{svg,png}
│  └─ diag3_interactions.{svg,png}
│  └─ ARCHITECTURE.pdf
│  └─ gen_pdf.js        # node docs/gen_pdf.js → ARCHITECTURE.pdf
└─ ARCHITECTURE.md
```

## Mise en place VSCode + Wrangler

### wrangler.toml

```
name = "wc2026"
main = "worker/worker.js"
compatibility_date = "2026-06-11"

[[kv_namespaces]]
binding = "STATS_KV"
id = "749700d94d114156a758b14e2e9b6587"

[triggers]
crons = ["*/30 * * * *", "0 */2 * * *", "10 */2 * * *", "20 */2 * * *"]
```

### Commandes utiles

```
wrangler deploy          # Deployer le worker
wrangler dev --local      # Dev local
wrangler tail            # Logs temps reel
wrangler secret put API_TOKEN_FD
```

## URLs de production

| Service             | URL                                               |
|---------------------|---------------------------------------------------|
| Dashboard (prod)    | https://wc2026.pages.dev                          |
| Dashboard (staging) | https://staging.wc2026.pages.dev                  |
| Worker              | https://wc2026.mguenoun.workers.dev               |
| Status pipeline     | https://wc2026.mguenoun.workers.dev/stats/status  |
| Classement joueurs  | https://wc2026.mguenoun.workers.dev/stats/players |
| Classement gardiens | https://wc2026.mguenoun.workers.dev/stats/keepers |

## Diagrammes d'architecture

```
PNG dans docs/ . Pour regenerer : node docs/gen_diagrams.js
```

Diagramme 1 — Composants et déploiement

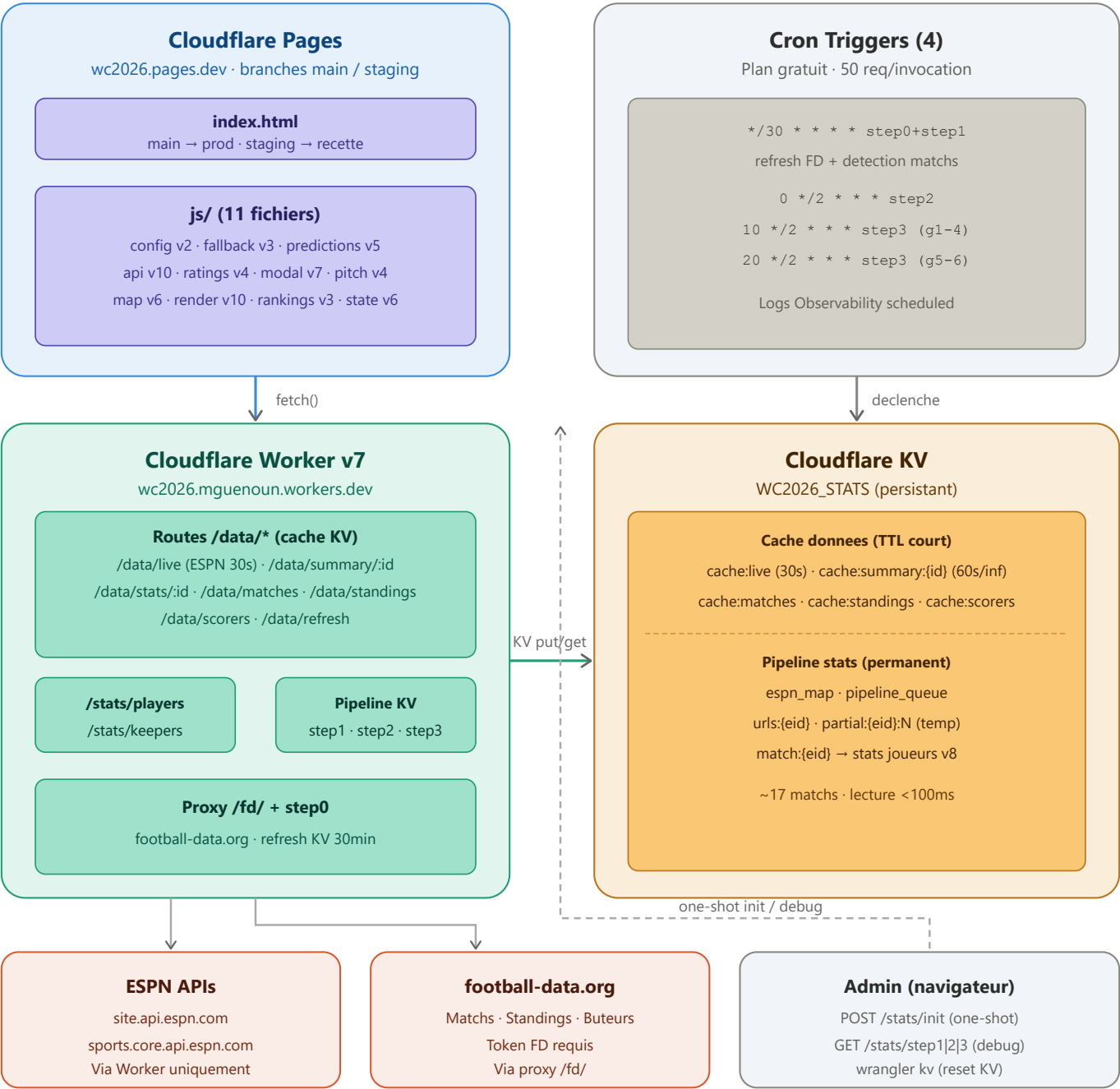


Diagramme 2 — Sequence au chargement de la page

Sequence au chargement de la page

Executes dans state.js (charge en dernier)

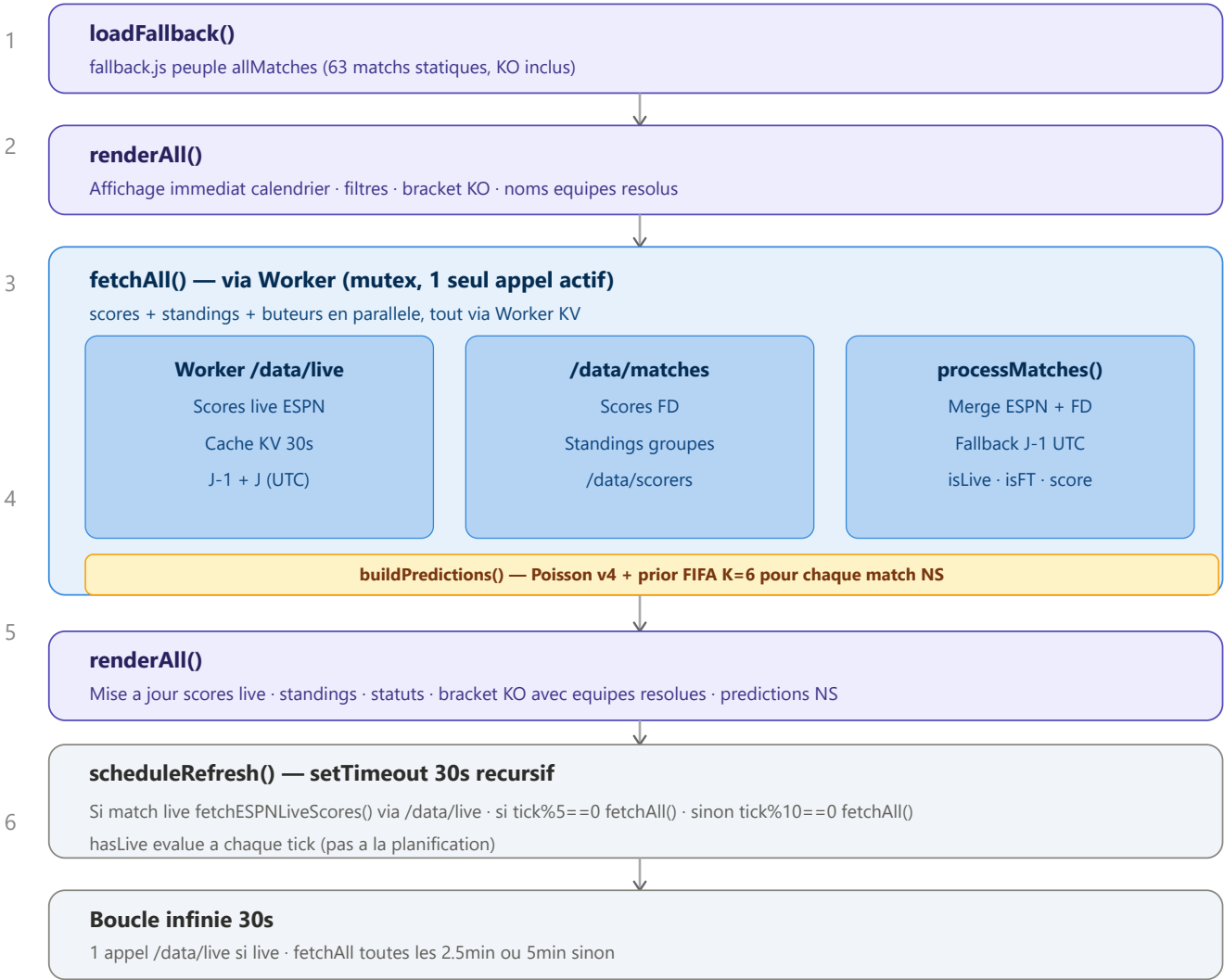


Diagramme 3 — Interactions UI par clic

